By mixing our encodings we get a false positive, because the UTF-16 encoding for U+C387 is *0xC3 0x87*, which matches the UTF-8 encoding for U+00C7. Now at this point, you might be thinking, "it's OK, we have `mb_substr_count()` and surely that can handle mixed encodings" and if that's the case, then sorry, but no. The C implementation of `mb_substr_count()` is essentially identical to `substr_count()`, with an added (faulty) attempt to normalize the encoding of both strings to UTF-8.

(For reference, the C source code for both functions can be found with the following links.)

https://github.com/php/php-src/blob/734f686ac4708abc1606a0661ce435d1a7851152/ext/mbstring/mbstring.c#L808

https://github.com/php/php-src/blob/f91dcad567379856d2883531fdf070ebaca0fd5f/ext/standard/string.c#L5685

```
$utf16Str = mb_convert_encoding("\u{C7}a\u{11F}ari", 'UTF-16');
$utf32Str = mb_convert_encoding("\u{C7}a\u{11F}ari", 'UTF-32');

var_dump(bin2hex($utf16Str));
var_dump(bin2hex($utf32Str));

var_dump(mb_substr_count($utf16Str, "\u{11F}ari"));
var_dump(mb_substr_count($utf32Str, "\u{11F}ari"));

Output:
string(24) "00c70061011f006100720069"
string(48) "000000c70000006100000011f000000610000007200000069"
int(0)
int(0)
```

As we can see, the encoding conversion works just fine; we get UTF-16 and UTF-32 encoded strings (all those null bytes…), but `mb_substr_count()` does not normalize them to UTF-8: if it did, then we would have non-zero returns. `mb_substr_count()` does accept a third parameter, allowing you to specify the encoding, but this applies to *both* arguments, and we're left with the same problem:

```
var_dump(mb_substr_count($utf16Str, "\u{11F}ari", 'UTF-16'));
var_dump(mb_substr_count($utf32Str, "\u{11F}ari", 'UTF-32'));
```